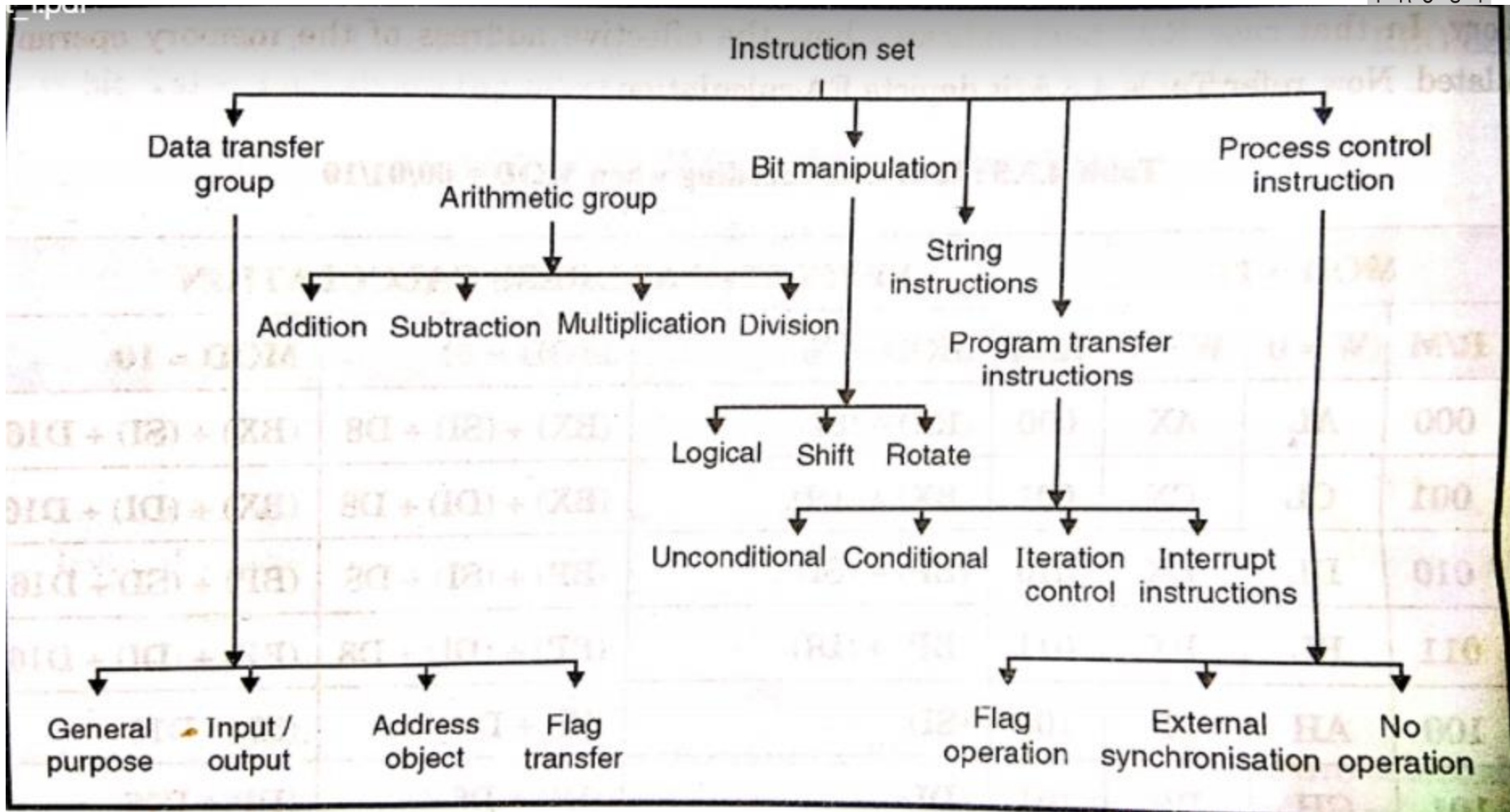


Instruction set of 8086

Compiled by
Mrs. Savita C. Raut

The Instruction set of 8086 microprocessor is classified into :

1. Data transfer instructions
2. Arithmetic instructions
3. Bit Manipulation instructions
4. String instructions
5. Program control transfer instructions
6. Process control instructions



Data transfer instructions

A. General purpose instructions:

- **MOV** – Used to copy the byte or word from the provided source to the provided destination.
- **PUSH** – Used to put a word at the top of the stack.
- **POP** – Used to get a word from the top of the stack to the provided location.
- **XCHG** – Used to exchange the data from two locations.
- **XLAT** – Used to translate a byte in AL using a table in the memory.

Data transfer instructions..cntd

B. Instructions for input and output port transfer

- **IN** – Used to read a byte or word from the provided port to the accumulator.
- **OUT** – Used to send out a byte or word from the accumulator to the provided port.

C. Instructions to transfer the address

- **LEA** – Used to load the address of operand into the provided register.
- **LDS** – Used to load DS register and other provided register from the memory
- **LES** – Used to load ES register and other provided register from the memory.

Data transfer instructions..cntd

D. Instructions to transfer flag registers

- **LAHF** – Used to load AH with the low byte of the flag register.
- **SAHF** – Used to store AH register to low byte of the flag register.
- **PUSHF** – Used to copy the flag register at the top of the stack.
- **POPF** – Used to copy a word at the top of the stack to the flag register.

MOV destination,Source

- Copies a **word or byte** from source to destination
- No flags are affected
- Any addressing mode is possible except implied.

Valid source and destination pairs

Destination	Source
Register	Register
Memory	register
Register	Memory
Register	Immediate
Memory	Immediate
Seg. Register	Register (16-bit)
Register (16-bit)	Seg. Register
Seg. Register	Memory
Memory	Seg. Register

can not set the value of CS and IP

MOV instruction

Note: Following data transfers are invalid

- Memory to Memory
- Destination can not be Immediate
- Segment register to Segment Register
- Immediate data to segment register

Examples:

Mov Ax,Bx

Mov Ax,[SI]

Mov [DI],Bx

Mov DS, Ax

Mov ES,[1234h] etc

Instruction :POP Destination

- This instruction copies a word from memory location pointed by SS:SP to the destination.
- When it is executed:
 - I. Data from memory location (in stack segment) SS:SP is first copied to the destination
 - II. And then SP is incremented by 2 ($SP=SP+2$)

Note : destination can be a general purpose register, Flag register ,segment register or memory location .

EX: POP AX

POP DS

Instruction :PUSH Source

When this instruction is executed:

- i) $SP = SP - 2$ (i.e. First SP is decremented by 2)
- ii) Copy the word from source register to the location in Stack Segment where SP points.

Note :1.Source operand (16-bit) can be a general purpose register,Flag register ,segment register or memory .

2. Addressing mode – register Indirect

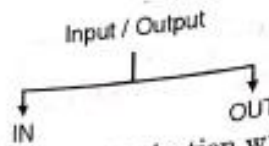
EX: PUSH Ax

PUSH BX

PUSH DS

Port related instructions: IN and OUT

Input/Output



These instructions are basically related to communication with I/O devices, mapped in I/O map.

1. IN : Copy data from a port

Mnemonic	IN accumulator, port address.
Algorithm	—

Flags	No flags are affected
Addr. Mode	Direct or Indirect port addressing mode

Operation	AX ← Contents of port or AL ← contents of port.	This instruction will copy data from a port whose address is given in the instruction to AX register or AL register. The data can be either 8 bit or 16 bit.
Example	IN AL, C8H.	This instruction will copy the contents of port whose address is C8H to the AL register.

- The IN instruction has two possible formats
 - (i) Fixed port (Direct port)
 - (ii) variable port (Indirect port)
- For the **Fixed port**, the port address is specified directly in the instruction. The port numbers are from 00 to FF i.e. 8 bit address is directly specified. Thus addressing mode is Direct port addressing. The above example is an example of direct port addressing. It can have only 8-bit address
- For the **Variable port** IN instruction, the port address is loaded into the DX register before the IN instruction. Since DX is a 16 bit register, the port address can be any number between 0000H and FFFFH. Therefore it is possible to address up to 65,536 ports in this mode.
 - e.g. MOV DX, 0FF0H; Initialize DX to point to port.
 - IN AL, DX; Input an 8 bit data from port 0FF0 to AL i.e. contents of port 0FF0 are copied to the AL register. The addressing mode is Indirect port addressing mode.

Port related instructions: IN and OUT

2. OUT Instruction - Output byte or word to a port

Mnemonic	OUT port address, Accumulator.	Flags	No flags are affected
Algorithm	—	Addr. Mode	Direct or Indirect port addressing mode
Operation	Contents of AX → Port address / Contents of AL → port address.	This Instruction copies a byte from AL or a word from AX to the specified port.	
Example	MOV DX, FFF8H OUT DX, AL	Load desired port address in DX. Copies contents of AL to given port address in register DX.	

- It has two possible forms.
 - i) Fixed port (Direct port)
 - ii) Variable port (Indirect port)
- In the **fixed port** form, the 8 bit port address is specified directly in the instruction with this form, any one of 256 possible ports can be addressed.
e.g. OUT 3B H, AL ; copies the contents of AL to port 3B H.
- The addressing mode is Direct port addressing mode.
- In the **variable port** form, the contents of AL or AX will be copied to the port at an address contained in DX. The DX register should be loaded with the desired port address. The above example in table is an example of indirect port addressing.

Conditional Transfer

Common Operations		
JC	JUMP if carry	CF = 1
JNC	JUMP if not carry	CF = 0
JZ/JE	JUMP if zero (or equal)	ZF = 1
JNZ/JNE	JUMP if not zero (or not equal)	ZF = 0
JP/JPE	JUMP if parity (or even parity)	PF = 1
JNP/JPO	JUMP if not parity (or odd parity)	PF = 0
JCXZ	JUMP if CX is zero	CX = 0000H
Signed Operations		
JO	JUMP if overflow	OF = 1
JNO	JUMP if not overflow	OF = 0
JS	JUMP if sign (- ve)	S = 1
JNS	JUMP if not sign (+ ve)	S = 0
JL/JNGE	JUMP if less (i.e. either greater nor equal)	SF $\oplus$ OF = 1
JNL/JGE	JUMP if not less (i.e. either greater or equal)	SF $\oplus$ OF = 0
JLE/JNG	JUMP if less or equal (i.e. not greater)	(SF $\oplus$ OF) + ZF = 1
JNLE/JG	JUMP if neither less nor equal (i.e. greater)	(SF $\oplus$ OF) + ZF = 0
Unsigned Operations		
JB/JNAE	JUMP if below (i.e. neither above nor equal)	CF = 1
JNB/JAE	JUMP if not below (i.e. either above or equal)	CF = 0
JBE/JNA	JUMP if below or equal (i.e. not above)	CF $\oplus$ ZF = 1
JNBE/JA	JUMP if neither below nor equal (i.e. above)	CF $\oplus$ ZF = 0

1. ADD – Add byte or word

Mnemonic : ADD destination, source

Algorithm : destination = destination + source

Operation : (Destination) $\leftarrow$ (destination) + (source)

This instruction adds a number from source to number from destination and puts the result to specified destination.

Sr. No.	Destination	Source
1.	Register	Register
2.	Register	Memory
3.	Memory	Register
4.	Register	Immediate
5.	Memory	Immediate
6.	Accumulator	Immediate

Note : (i) Both the operands i.e. source and destination, cannot be memory locations.

e.g. ADD [1234], [5678] $\Rightarrow$ Incorrect

String related instructions

REP	Repeat
REPE/REPZ	Repeat while equal/zero
REPNE/REPNZ	Repeat while not equal/not zero
MOVSX/MOVSQ	Move byte or word string
CMPSX/CMPSQ	Compare byte or word string
SCASX/SCASQ	Scan byte or word string
LODSX/LODQ	Load byte or word string
STOSX/STOQ	Store byte or word string

String instructions..cntd

MOVS_B/MOV_{SW}

In case of byte :

- $ES : [DI] = DS : [SI]$
- If $DF = 0$ then
 $SI = SI + 1$
 $DI = DI + 1$

else

$$SI = SI - 1$$
$$DI = DI - 1$$

In case of word :

- $ES : [DI] = DS : [SI]$
- If $DF = 0$ then,
 $SI = SI + 2$
 $DI = DI + 2$

else

$$SI = SI - 2$$
$$DI = DI - 2$$

- This instruction copies a byte/word from a location in Data Segment to a location in Extra Segment

String instructions..cntd

CMPSB/CMPSW

Two alternate mnemonics for the same instruction

CMPB, (compare string byte)

CMPW, (compare string word)

For byte operation

DS : [SI] - ES : [DI]

Set flags according to result

→ OF, SF, ZF, AF, PF, CF.

If DF = 0 then

SI = SI + 1

DI = DI + 1

else

SI = SI - 1

DI = DI - 1

For word operation

DS : [SI] - ES : [DI]

Set flags, according to result

→ OF, SF, ZF, AF, PF, CF.

If DF = 0 then

SI = SI + 2

DI = DI + 2

else

SI = SI - 2

DI = DI - 2

- This instruction compares a byte/word from a location in Data Segment to a location in Extr
- a Segment

String instructions..cntd

3. SCAS/SCASB/SCASW

(Scan a string Byte or Word)

Mnemonic	SCASB/SCASW SCASB (for byte operation) Or SCASW (for word operation)	Flags	No flags are affected.
Algorithm	For SCASB • AL – ES : [DI] • Set flags according to result • If DF = 0 then DI = DI + 1 For SCASW • AX – ES : [DI] • Set flags according to result • If DF = 0 then DI = DI + 2 - Else DI = DI – 2	Addr. Mode	String addressing mode
Operation	• This instruction compares a byte in AL or word in AX with a byte or word pointed to by DI in ES. Hence the string to be scanned must be in ES, and offset in DI. After comparison DI will be automatically incremented or decremented according to direction flag DF.		

String instructions..cntd

LODS

(Load string byte in AL or string word into AX.)

Mnemonic	LODSB/LODSW Or LODSB (For byte operation) Or LODSW (For word operation)
Algorithm	LODSB • $AL = DS : [SI]$ If $DF = 0$ then $SI = SI + 1$ else $SI = SI - 1$ LODSW • $AX = DS : [SI]$ If $DF = 0$ then $SI = SI + 2$ Else $SI = SI - 2$

Flags	No flags are affected.
Addr. Mode	String addressing mode

Operation	This instruction copies a byte from a string location pointed to by SI to AX.
-----------	---

String instructions..cntd

STOS/STOSB/STOSW
 (Store byte or word in string)

Mnemonic	STOSB/STOSW STOSB (For byte operation) or STOSW (for word operation)
Algorithm	For STOSB • $ES : [DI] = AL$ If $DF = 0$ then $DI = DI + 1$ else $DI = DI - 1$ For STOSW • $ES : [DI] = AX$ If $DF = 0$ then $DI = DI + 2$ else $DI = DI - 2$

Flags	No flags are affected
Addr. Mode	String addressing mode

Operation	It transfers (copies) a byte or word from register AL or AX to the string element addressed in ES by DI. Depending on DF flag, DI is automatically incremented or decremented.
------------------	--

String instructions-Summary

STOSB AL $\Rightarrow$ ES:[DI]

After string opr, if DF = 0, then DI is incremented by 1 and viceversa

LODSB AL $\Leftarrow$ DS:[SI]

After string opr, if DF = 0, then SI is incremented by 1 and viceversa

MOVSB DS:[SI] $\Rightarrow$ ES:[DI]

After string opr, if DF = 0, then SI and DI is incremented by 1 and viceversa

SCASB AL – ES:[DI]

After string opr, if DF = 0, then DI is incremented by 1 and viceversa

CMPSB DS:[SI] – ES:[DI]

After string opr, if DF = 0, then SI and DI is incremented by 1 and viceversa

Prefix:REP

String instructions can handle at a time one byte or one word, but to handle large block of data, REP prefix is used with CX as a counter

Types

REP

dec CX by 1 and repeat that string instruction if CX is non zero. Otherwise control continues sequentially

REPNE/REPNZ

dec CX by 1 and repeat that string instruction if CX is non zero AND if that string inst. Produced non – zero result. Otherwise control continues sequentially

REPE/REPZ

dec CX by 1 and repeat that string instruction if CX is non zero AND if that string inst. Produced zero result. Otherwise control continues sequentially

LOOP Instruction

It is used to execute series of instructions for a given number of times and that number of times is stored in CX

Types

LOOP AGAIN

dec CX by 1 and transfer control to the instruction having label AGAIN if CX is non zero. Otherwise control continues sequentially

LOOPNE/LOOPNZ

dec CX by 1 and transfer control to the instruction having label AGAIN if CX is non zero AND if previous instruction produced non zero result. Otherwise control continues sequentially

LOOPE/LOOPZ

dec CX by 1 and transfer control to the instruction having label AGAIN if CX is non zero AND if previous instruction produced zero result. Otherwise control continues sequentially

Sample Program:
Problem statement – clear the
contents of three memory location

<i>Memory location</i>	<i>content</i>
62380	00
62381	00
62382	00

Using MOV instruction

MOV AX,6000H *move 6000h into AX*

MOV DS,AX *move AX contents into DS*

MOV DI,2380H *move 2380H into DI*

MOV CX,0003H *move 0003H into CX*

MOV AL,00H *move 00H into AL*

Again MOV [DI],AL *AL => DS:[DI]*

INC DI *Increment the contents of DI*

DEC CX

JNZ AGAIN *dec CX, and if CX is NZ go to again*

HLT *Halt*

Using Loop instruction

MOV AX,6000H *move 6000h into AX*

MOV DS,AX *move AX contents into DS*

MOV DI,2380H *move 2380H into DI*

MOV CX,0003H *move 0003H into CX*

MOV AL,00H *move 00H into AL*

Again MOV [DI],AL AL => DS:[DI]

INC DI Increment the contents of DI

LOOP AGAIN dec CX, and if CX is NZ go to again

HLT Halt

Using string instructions (Use of Prefix)

MOV AX,6000H *move 6000h into AX*

MOV ES,AX *move AX contents into ES*

MOV DI,2380H *move 2380H into DI*

CLD *clear the direction flag*

MOV CX,0003H *move 0003H into CX*

MOV AL,00H *move 00H into AL*

REP STOSB *AL => ES:[DI]and increment DI
(rep this CX times)*

HLT *Halt*

Prog 2: Problem statement block transfer

Transfer contents of 52500 to 62500 and 52501 to
62501

Block transfer

MOV AX,5000 H

MOV DS,AX

MOV AX,6000H

MOV ES,AX

MOV SI,2500H

MOV DI,2500H

CLD

MOV CX,0002H

REP MOVSB

*DS:[SI] =>ES:[DI] and increment DI
and SI (rep this CX times)*

HLT

FIND THE GIVEN BYTE 03 IN AN ARRAY

```

MOV AX,2000H
MOV ES,AX
MOV DI,0500H
MOV CX,0006
MOV AL, 03
CLD
REPNE SCASB
DEC DI
hlt
  
```

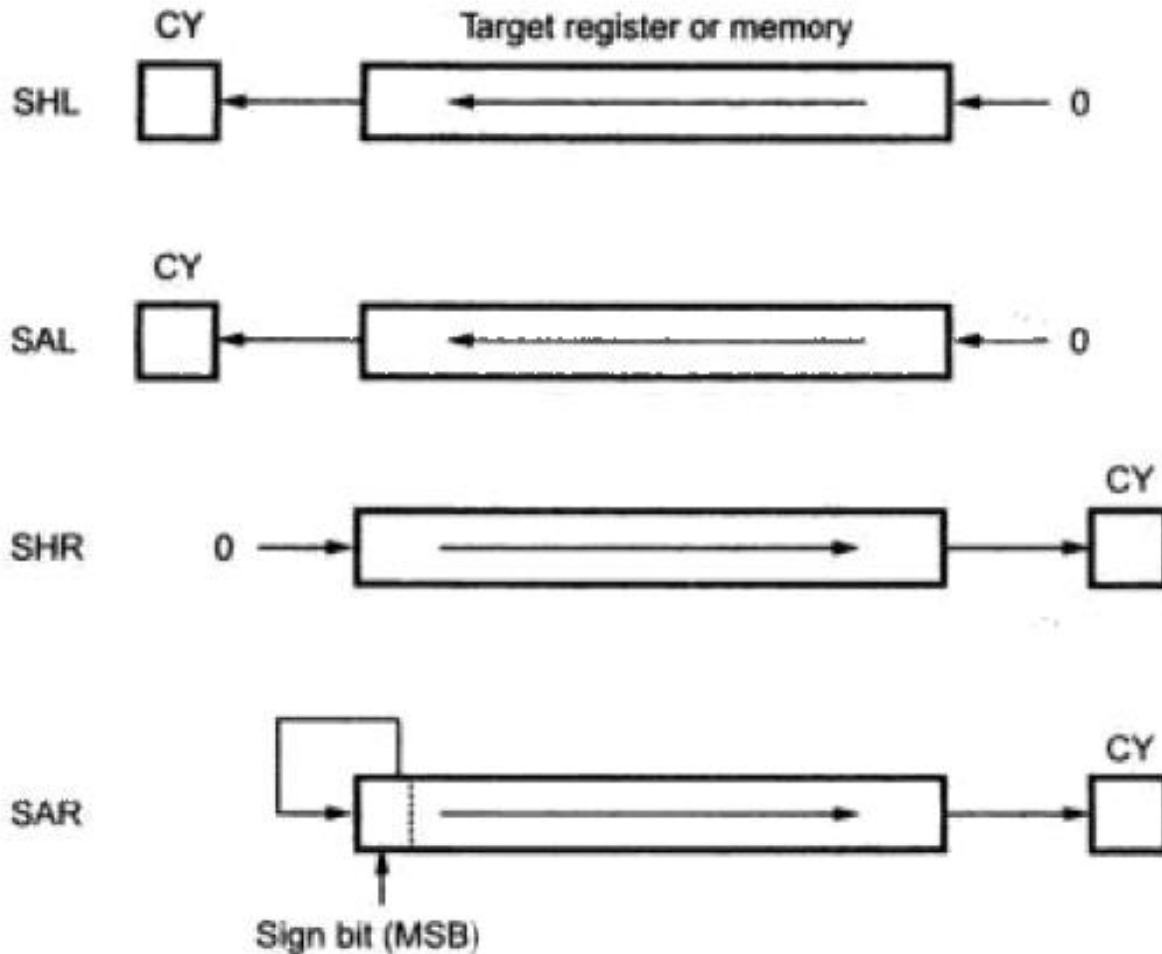
Memory location	Content
2000:0500	78 h
2000:0501	19 h
2000:0502	65 h
2000:0503	45 h
2000:0504	03 h
2000:0505	32 h
2000:0506	13 h
2000:0507	26 h

Shift Instructions

- Shift instructions move the binary data to the left or right by shifting them within the register or memory location.
- They also can perform multiplication of powers of 2^{+n} and division of powers of 2^{-n}
- There are two type of shifts logical shifting and arithmetic shifting.
- Arithmetic shifting is used with signed numbers while logical shifting with unsigned numbers.

Shift instruction

General Format:
SHR destination, count



Example:

MOV BL, B7H ; BL is made B7H

SAL BL, 1 ; shift the content of BL register one place to left. Before

execution,

CY	B7	B6	B5	B4	B3	B2	B1	B0
0	1	0	1	1	0	1	1	1

After the
execution,

CY	B7	B6	B5	B4	B3	B2	B1	B0
1	0	1	1	0	1	1	1	0

SHR instruction

This instruction shifts each bit in the specified destination to the right and 0 is stored in the MSB position. The LSB is shifted into the carry flag. The destination can be of byte size or of word size, also it can be a register or a memory location. Number of shifts is indicated by the count.

All flags are affected

General Format:

SHR destination, count

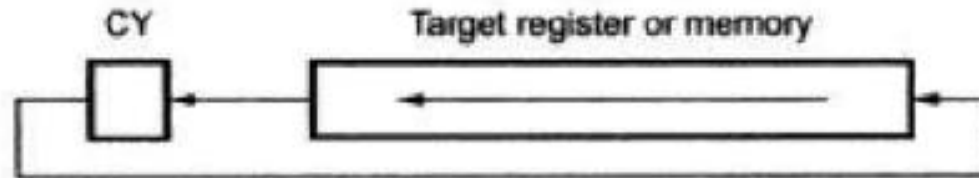
Example:

MOV BL, B7H ; BL is made B7H

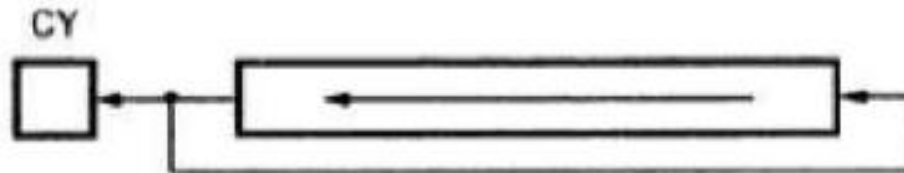
SHR BL, 1 ; shift the content of BL register one place to the right.

Rotate Instructions

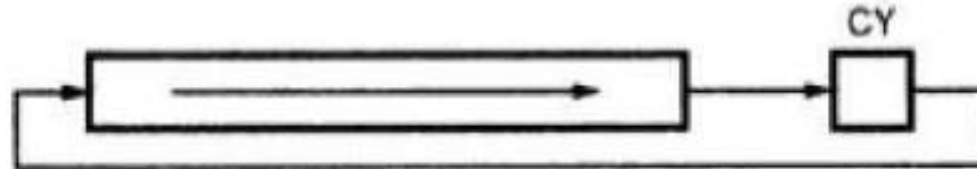
RCL
(Rotate left through carry)



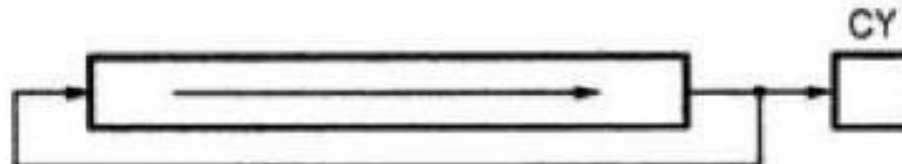
ROL
(Rotate left)



RCR
(Rotate right through carry)



ROR
(Rotate right)



ROR – ROTATE RIGHT (LSB INTO CARRY AND MSB AND OTHER BITS SHIFT TO RIGHT)

ROL- ROTATE LEFT (MSB INTO CARRY AND LSB AND OTHER BITS SHIFT TO LEFT)

RCR - ROTATE RIGHT THRU CARRY (LSB INTO CARRY AND CARRY INTO MSB AND OTHER BITS SHIFT TO RIGHT)

RCL - ROTATE LEFT THRU CARRY (MSB INTO CARRY AND CARRY INTO LSB AND OTHER BITS SHIFT TO LEFT)

FORMAT FOR EXAMPLE:

RCR BX,01

RCR BX,CL (If shifting is to be done more than once, count has to be stored in CL)

Program: to count the Zero ,positive, negative numbers

```
MOV AX, 0000
MOV DS, AX
MOV BX, D000
MOV CL, [BX]
MOV DX, 0000
MOV AH, 00
UP: INC BX
MOV AL, [BX]
ADD AL, 00
JNZ DOWN1
INC DH
JMP DOWN 3
DOWN1: RCL AL, 01
JNC DOWN2
INC AH
JMP DOWN3
DOWN2: INC DL
DOWN3: DEC CL
JNZ UP
INT 3
```

If MSB =1,then Negative no.

If MSB=0 ,then positive no.

Reg Dh -- Zeros

Reg Dl – positive no.s

Reg Ah-negative no.s

WAP to count the number of positive and negative bytes stored at m.l. 10500H (for 10 bytes)

```

MOV DX, 1060H
MOV DS, DX
MOV DI, 0500H
MOV CL, 0AH ; count for total bytes
MOV DX, 0000H ; DH=00 ; DL=00H
                ; (+ve)    (-ve)

NEXT: MOV AL, [DI]
      ROL AL, 01H
      JNC positive
      INC DL
      JMP GO
positive: INC DI
GO:      INC DI
      DEC CL
      JNZ NEXT
      INT 03H
    
```

if no. is +ve
if no. is -ve